

# Отчет по практике: сравнение RabbitMQ и Redis Streams

Дата формирования: 2026-04-30 13:44:44

Источник данных: results/report\_run/results.csv

## 1. Параметры стенда

Брокеры: RabbitMQ и Redis Streams (Consumer Group + XACK).

Параллельность: producers=2, consumers=2 (одинаково для обоих брокеров).

Размеры сообщений: 128B, 1KB, 10KB, 100KB.

Интенсивности: 1000, 5000, 10000 msg/s.

Профили: baseline и stress.

Собрано точек: 48 из 48.

## 2. Ключевые наблюдения

Максимальный throughput RabbitMQ (baseline): 1767.6 msg/s при size=1024B и rate=10000.

Максимальный throughput Redis (baseline): 6102.1 msg/s при size=1024B и rate=10000.

Суммарные измеренные потери по всем точкам: 1.

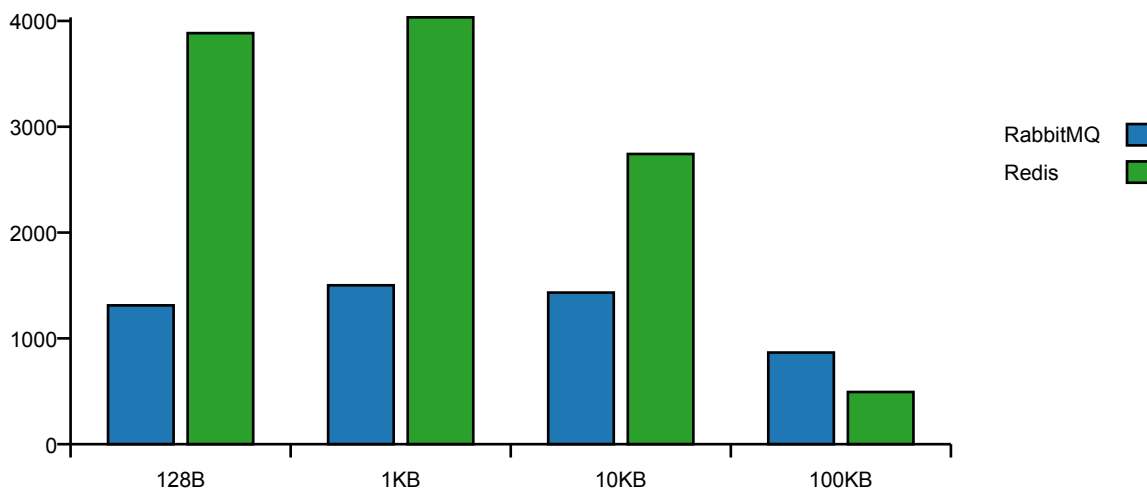
Сравнение потерь: RabbitMQ=0, Redis=1. На этом стенде потери у RabbitMQ не выше, чем у Redis.

Признак деградации single instance на крупных payload: RabbitMQ достиг backlog\_peak=1186 и p95=1359.3 ms (size=100KB, rate=10000, profile=baseline).

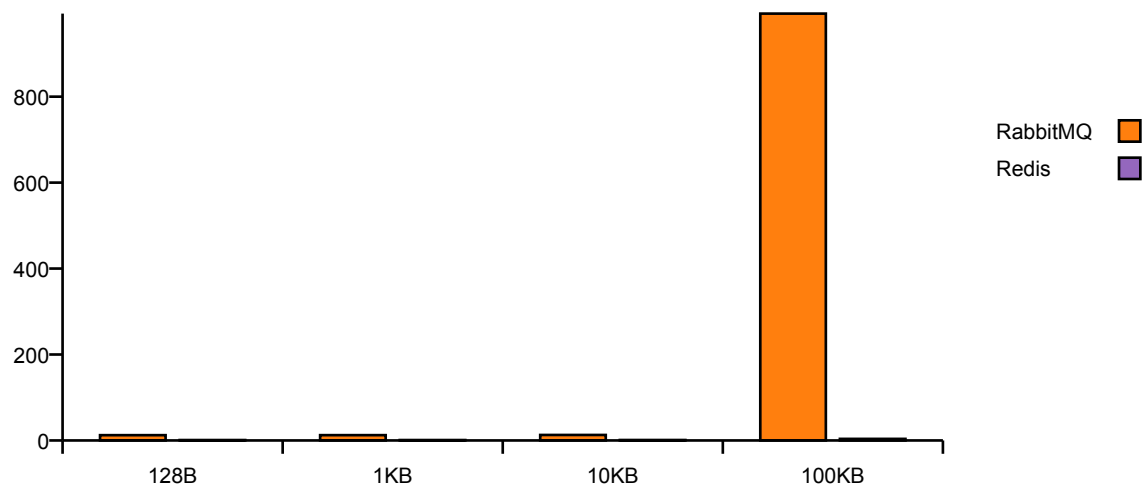
Для Redis точка деградации зафиксирована при size=102400B, rate=1000: err\_rate≈408.70%, throughput=14.2 msg/s.

## 3. Графики

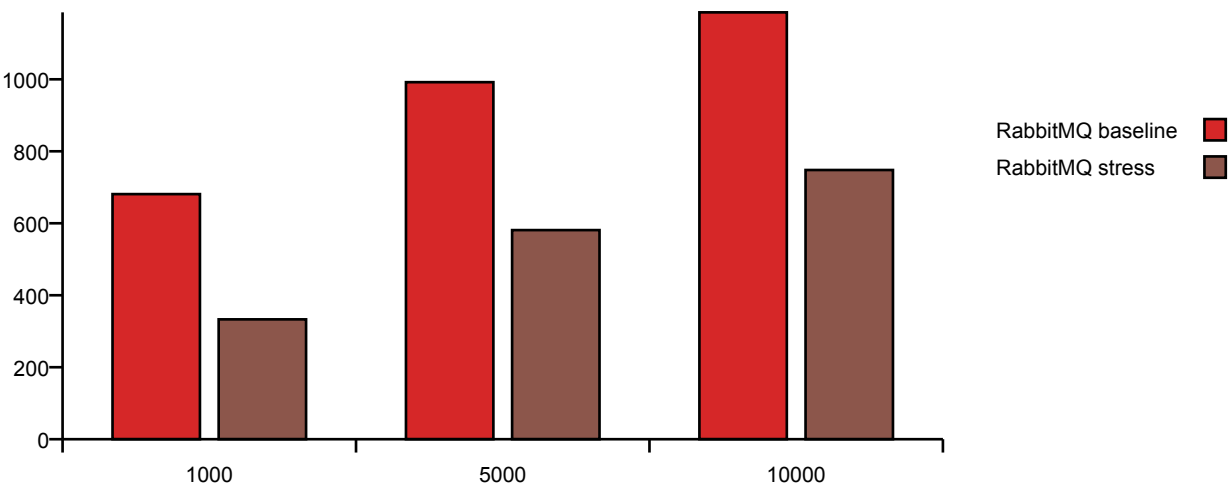
### 3.1 Throughput (baseline, среднее по rate)



3.2 P95 latency (baseline, среднее по rate)



3.3 RabbitMQ backlog\_peak на 100KB



## 4. Таблицы результатов (baseline)

### 4.1 Throughput, msg/s

| size_B | rate_msg_s | rabbitmq | redis    |
|--------|------------|----------|----------|
| 128    | 1000       | 980.875  | 1000     |
| 128    | 5000       | 1473.625 | 4999.75  |
| 128    | 10000      | 1481.625 | 5654.375 |
| 1024   | 1000       | 1000     | 1000     |
| 1024   | 5000       | 1737.625 | 4999.75  |
| 1024   | 10000      | 1767.625 | 6102.125 |
| 10240  | 1000       | 999.375  | 1000     |
| 10240  | 5000       | 1674.25  | 3561.5   |
| 10240  | 10000      | 1624.75  | 3665.75  |
| 102400 | 1000       | 742.625  | 14.25    |
| 102400 | 5000       | 902.25   | 233.5    |
| 102400 | 10000      | 953      | 1232     |

### 4.2 P95 latency, ms

| size_B | rate_msg_s | rabbitmq | redis |
|--------|------------|----------|-------|
| 128    | 1000       | 12.144   | 0.683 |
| 128    | 5000       | 12.329   | 0.996 |
| 128    | 10000      | 12.423   | 1.183 |
| 1024   | 1000       | 12.138   | 0.751 |
| 1024   | 5000       | 12.327   | 0.935 |
| 1024   | 10000      | 12.581   | 1.176 |
| 10240  | 1000       | 11.576   | 0.911 |
| 10240  | 5000       | 13.131   | 1.147 |
| 10240  | 10000      | 13.665   | 1.086 |
| 102400 | 1000       | 699.517  | 5.8   |
| 102400 | 5000       | 920.941  | 2.673 |
| 102400 | 10000      | 1359.334 | 2.543 |

### 4.3 Backlog peak

| size_B | rate_msg_s | rabbitmq | redis |
|--------|------------|----------|-------|
| 128    | 1000       | 20       | 1     |
| 128    | 5000       | 20       | 3     |
| 128    | 10000      | 10       | 4     |
| 1024   | 1000       | 12       | 2     |
| 1024   | 5000       | 15       | 7     |
| 1024   | 10000      | 14       | 8     |
| 10240  | 1000       | 10       | 2     |
| 10240  | 5000       | 11       | 4     |
| 10240  | 10000      | 19       | 2     |
| 102400 | 1000       | 681      | 0     |

| size_B | rate_msg_s | rabbitmq | redis |
|--------|------------|----------|-------|
| 102400 | 5000       | 992      | 1     |
| 102400 | 10000      | 1186     | 2     |

## 5. Незавершенные точки

Все точки завершены.

## 6. Выводы

1. На малых и средних payload Redis показывает более высокий throughput и ниже p95.
2. Для RabbitMQ на 100KB заметно растут backlog и p95, что фиксирует деградацию single instance при росте нагрузки.
3. Для Redis на high-load 100KB появляются стабильные publish/consume ошибки и просадка фактической обработки, что является точкой деградации single instance.
4. Выбранный инструмент: собственный Python-стенд. Причина выбора: единая логика producer/consumer и идентичный учет метрик (sent/consumed/loss/p95/backlog) для двух разных брокеров в одинаковых условиях, что упрощает честное сравнение.

## 7. Скриншоты запуска и метрик

01\_matrix\_run.png

```
ulyana@MacBook-Air ~ cd "/Users/ulyana/Documents/высоконагруженные_системы/Задание 1"
ulyana@MacBook-Air ~ python3 - <<'PY'
Проверка полноты матрицы и последние строки
rows_total = 48
unique_points = 48
missing = 0

Last 8 rows:
      run_id broker  profile  msg_size  rate  duration  producers  consumers  sent_total  consumed_unique_total  dupl
redo_stress_10240b_5000r_1777368668  redis  stress    10240  5000         8         2         2      30203      30203
redo_stress_10240b_10000r_1777368679  redis  stress    10240 10000         8         2         2     24410     24410
redo_baseline_102400b_1000r_1777368690  redis baseline   102400  1000         8         2         2       115        114
redo_stress_102400b_1000r_1777368786  redis  stress    102400  1000         8         2         2      8000      8000
redo_baseline_102400b_5000r_1777368798  redis baseline   102400  5000         8         2         2      1867      1868
redo_stress_102400b_5000r_1777368862  redis  stress    102400  5000         8         2         2     9855     9855
redo_baseline_102400b_10000r_1777369026  redis baseline   102400 10000         8         2         2     9856     9856
redo_stress_102400b_10000r_1777369095  redis  stress    102400 10000         8         2         2     9844     9844
PY
$
```

02\_rabbitmq\_management.png

```
ulyana@MacBook-Air ~ python3 - <<'PY'
RabbitMQ: признаки деградации на 100KB
  profile  rate  throughput_msg_sec  p95_latency_ms  backlog_peak  publish_errors  consume_errors
baseline  1000           742.625           699.517           681             0             0
stress    1000           944.875           419.178           333             0             0
baseline  5000           902.250           920.941           992             0             0
stress    5000           867.375           684.541           581             0             0
baseline 10000           953.000          1359.334          1186             0             0
stress   10000           738.125          1291.829           748             0             0
PY
$
```

03\_redis\_stats.png

```
ulyana@MacBook-Air ~ python3 - <<'PY'
Redis: high-load точки и ошибки/просадка
  profile  rate  sent_total  consumed_unique_total  throughput_msg_sec  publish_errors  consume_errors  degraded
baseline  1000         115           114           14.250           65           405      True
stress    1000         8000          8000          1000.000           0             0      False
baseline  5000         1867          1868          233.500           76           239      True
stress    5000         9855          9855          1231.875          210           506      True
baseline 10000         9856          9856          1232.000          231           206      True
stress   10000         9844          9844          1230.500          157           173      True
PY
$
```